

Механизмы цифрового забвения

Технический отчёт и руководство пользователя. Проект 2039, МИЭМ НИУ ВШЭ

Команда проекта 2039 — МИЭМ НИУ ВШЭ, ТК 26

Апрель 2026

Contents

1	Аннотация	3
2	Введение	4
2.1	Контекст и актуальность	4
2.2	Цели и задачи проекта	4
2.3	Область применения	4
3	Часть I. Теоретическая база	5
3.1	Терминология и обозначения	5
3.2	Постановка задачи	5
3.3	Обзор подходов к Machine Unlearning	5
3.3.1	Классификация методов	5
3.3.2	Почему выбран SISA	6
3.4	Архитектура SISA	6
3.4.1	Построение	6
3.4.2	Процедура разобучения	6
3.4.3	Стабильность маршрутизации	7
3.5	Базовые модели обучения	7
3.5.1	Логистическая регрессия (PyTorch)	7
3.5.2	Случайный лес (scikit-learn)	7
3.6	Метрики оценки	7
3.6.1	Классификационные метрики	7
3.6.2	Метрики качества разобучения	8
3.6.3	Методики верификации (вне score прототипа)	8
3.7	Генерация синтетических данных	8
3.7.1	Поддерживаемые распределения	8
3.7.2	Корреляционные структуры	8
3.7.3	Генераторы персональных данных	8
3.7.4	Режимы разметки	8
3.7.5	Валидация сгенерированных данных	8
4	Часть II. Архитектура прототипа	9
4.1	Структура репозитория	9
4.2	Поток данных	9
4.3	Ключевые классы	10
5	Часть III. Единая интерактивная панель panel.py	11
5.1	Назначение	11
5.2	Запуск	11

5.3	Основные команды	11
5.3.1	(1) Генерация датасета	11
5.3.2	(5) Обучение SISA-ансамбля	11
5.3.3	(7) Разобучение по user_id	12
5.3.4	(8) Разобучение по фильтру	12
5.3.5	(10) Отчёт по запуску	12
5.3.6	(0) Полный цикл	12
5.4	Поведение интерфейса	12
5.5	Состояние сессии	12
5.6	Артефакты	13
6	Часть IV. Руководство пользователя	14
6.1	Установка	14
6.2	Типовые сценарии	14
6.2.1	Сценарий А. Полный цикл через единую панель (рекомендуется)	14
6.2.2	Сценарий Б. Демо-прогон одной командой	14
6.2.3	Сценарий В. Работа с существующими CSV	14
6.2.4	Сценарий Г. Батч-режим через отдельные CLI	14
6.3	Работа из Python-кода	15
6.4	Формат конфигурации	15
6.5	Интерпретация метрик	16
6.6	Повышение точности после разобучения	16
6.6.1	Параметрические меры (без изменений кода)	16
6.6.2	Архитектурные улучшения	17
6.6.3	Дисциплина запросов	17
7	Часть V. Результаты и аудит	18
7.1	Тестовое покрытие	18
7.2	Эмпирические результаты	18
7.3	Аудит кода — свод исправлений по двум итерациям	18
7.3.1	Итерация 1 (до первого отчёта)	18
7.3.2	Итерация 2 (после добавления panel.py)	19
7.4	Документированные ограничения	20
8	Часть VI. Юридический контекст	21
8.1	Федеральный закон № 152-ФЗ, статья 14	21
8.2	GDPR, статья 17	21
8.3	149-ФЗ, изменения 2024 года	21
9	Заключение	22
10	Литература	23

1 Аннотация

В настоящем документе представлен прототип системы исследования и реализации *механизмов цифрового забвения* (Digital Forgetting, Machine Unlearning) — технологии безопасного удаления влияния персональных данных на обученные модели машинного обучения. Прототип разработан в рамках проекта 2039 «Механизмы цифрового забвения: анализ и моделирование с применением методов машинного обучения», ВШЭ МИЭМ, по заказу Технического комитета по стандартизации «ТК 26», в рамках национального проекта «Экономика данных и цифровая трансформация государства».

Система построена вокруг архитектуры **SISA** (*Sharded, Isolated, Sliced, Aggregated*) — одного из двух признанных подходов к точному (exact) машинному разобучению. Реализованы: генератор синтетических табличных данных с контролируемыми распределениями и корреляциями, два базовых семейства моделей (PyTorch-логистическая регрессия и случайный лес из scikit-learn), процедуры **delete-by-ID** и **delete-by-filter** с переобучением только затронутых шардов, полный набор метрик оценки качества и эффективности разобучения, а также **единая интерактивная CLI-панель** (panel.py), объединяющая все стадии эксперимента в одном процессе. Инструментарий сопровождается детерминированным логированием, YAML-конфигурацией экспериментов и воспроизводимыми артефактами запусков, что удовлетворяет требованиям Федерального закона от 27.07.2006 № 152-ФЗ «О персональных данных», ст. 14, и Общего регламента ЕС по защите данных (GDPR), ст. 17.

По сравнению с первой версией отчёта добавлены: описание единой панели управления, расширенное руководство пользователя, сводная таблица багов, обнаруженных во время двух итераций аудита кода, и полная спецификация интерфейса.

2 Введение

2.1 Контекст и актуальность

В современную эпоху цифровизации существует острая потребность в защите персональных данных и реализации права граждан на их удаление. Статья 17 GDPR («право на забвение») и статья 14 Федерального закона № 152-ФЗ предоставляют физическим лицам право требовать удаления своих данных, если они более не нужны для заявленных целей обработки или если согласие было отозвано.

Технически корректная реализация этих требований в системах машинного обучения представляет нетривиальную задачу: после того как конфиденциальная запись поучаствовала в обучении модели, её влияние остаётся «размазанным» по миллионам параметров. Простое удаление строки из обучающего набора не гарантирует, что модель не сможет затем выдать эту информацию через предсказания или косвенные статистические выводы — такие как атаки *membership inference* (Shokri и соавт., 2017).

2.2 Цели и задачи проекта

Цель проекта. Комплексное исследование механизмов цифрового забвения с применением методов машинного обучения, включая: (i) разработку генератора синтетических данных; (ii) реализацию алгоритмов machine unlearning; (iii) создание прототипа системы с интерфейсом управления экспериментами.

Задачи.

1. Анализ правовых норм (152-ФЗ, GDPR) и существующих подходов к Machine Unlearning.
2. Разработка генератора синтетических данных с настраиваемыми распределениями признаков, корреляциями и разметкой (Python, NumPy, Pandas).
3. Обучение базовых моделей машинного обучения (логистическая регрессия, случайный лес).
4. Реализация механизма удаления данных и пересчёта метрик качества.
5. Интеграция компонентов и тестирование прототипа.
6. Документирование и анализ результатов.

2.3 Область применения

Разработанный прототип предназначен для использования:

- в государственных и муниципальных информационных системах для соответствия 152-ФЗ;
- в финансово-кредитных учреждениях для безопасного удаления клиентских данных;
- в здравоохранении для управления медицинскими данными в соответствии с законодательством;
- в образовательных и исследовательских учреждениях для обучения специалистов в области защиты данных;
- в корпоративных системах управления данными.

3 Часть I. Теоретическая база

3.1 Терминология и обозначения

- **Цифровое забвение (ЦЗ)** — процесс удаления или маскировки персональных данных и цифровых следов пользователя в информационных системах.
- **Право на забвение (Right to be Forgotten, RTBF)** — юридическое право субъекта данных требовать удаления персональной информации.
- **Machine Unlearning (MU)** — направление ML, целью которого является удаление влияния определённого подмножества обучающих примеров на обученную модель без необходимости переобучения с нуля.
- **Exact unlearning** — разобучение, дающее модель, статистически неотличимую от переобучения с нуля на очищенных данных.
- **Approximate / certified unlearning** — разобучение с формальной гарантией (ε, δ) -близости распределений выходов.
- **Синтетические данные** — искусственно созданные данные, имитирующие статистические характеристики реальных без прямого копирования записей.
- **SISA** — архитектура разобучаемого ансамбля: *Sharded, Isolated, Sliced, Aggregated* (Bourtoule и соавт., 2021).
- **Прогнозное расхождение (prediction disagreement)** — доля тестовых объектов, на которых жёсткая метка изменилась после разобучения.
- **MIA (Membership Inference Attack)** — атака, пытающаяся определить, была ли конкретная запись использована при обучении модели (Shokri и соавт., 2017).

3.2 Постановка задачи

Пусть задан обучающий набор $D = \{(x_i, y_i)\}_{i=1}^N$, где $x_i \in \mathbb{R}^d$ — признаковое описание, $y_i \in \{0, 1\}$ — целевая метка. Алгоритм обучения $\mathcal{A}$ отображает данные в модель: $\mathcal{A}(D) = h$.

Пусть пользователь запрашивает удаление подмножества $D_f \subset D$ (*forget set*). Идеальная процедура разобучения $\mathcal{U}$ должна удовлетворять:

$$\mathcal{U}(\mathcal{A}(D), D_f, D) \stackrel{d}{\approx} \mathcal{A}(D \setminus D_f),$$

то есть её выход статистически неотличим от модели, обученной с нуля на очищенном наборе. В определении Гуо и соавт. (2020) эта неотличимость формализуется в терминах дифференциальной приватности:

$$e^{-\varepsilon} \leq \frac{\Pr[\mathcal{U}(\mathcal{A}(D), D_f, D) \in S]}{\Pr[\mathcal{A}(D \setminus D_f) \in S]} \leq e^{\varepsilon} + \delta \quad \forall S \subseteq \mathcal{H},$$

откуда понятие (ε, δ) -certified unlearning.

В настоящем прототипе мы реализуем **exact unlearning через SISA**: равенство распределений достигается конструктивно — изоляцией обучения по шардам и переобучением *только* тех шардов, которые содержали удалённые записи.

3.3 Обзор подходов к Machine Unlearning

3.3.1 Классификация методов

Подход	Гарантия	Типичная стоимость	Область
Наивный retrain	Exact	$O(N \cdot d \cdot E)$	Универсально, но дорого
SISA (Bourtoule, 2021)	Exact	$O(D_s)$ на удаление	Любая ML-задача
Certified (Guo, 2020)	(ε, δ)	$O(d^2) + O(D_f d)$	Выпуклые потери

Подход	Гарантия	Типичная стоимость	Область
Amnesiac (Graves, 2021)	Exact (при хранении ΔW)	$O(E \cdot D_f)$	Нейронные сети
First-order (Warnecke, 2022)	Approx	$O(D_f d)$	Дифф. модели
DP-SGD (Abadi, 2016)	(ε, δ)	постоянная	Любые дифф. модели

3.3.2 Почему выбран SISА

1. **Универсальность.** Метод не требует выпуклости функции потерь и не зависит от конкретной архитектуры модели — достаточно, чтобы каждый шард обучался алгоритмом, поддерживающим `fit()`.
2. **Точность.** Даёт *строго* exact unlearning: остальные шарды остаются бит-в-бит идентичными.
3. **Реализуемость в обучающей среде.** Не требует дополнительного хранения градиентов за всю историю обучения (в отличие от Amnesiac) и не требует выпуклости (в отличие от Certified через Influence Function).
4. **Масштабируемость.** Стоимость разобучения пропорциональна *размеру одного шарда*, а не всего датасета.

3.4 Архитектура SISА

3.4.1 Построение

Дано: обучающий набор $D = \{(x_i, y_i)\}_{i=1}^N$, число шардов S и стабильный хеш $H : \text{UID} \rightarrow \{0, \dots, S-1\}$.

Шаг 1 (Sharded). Разбиваем D на дизъюнктные шарды по хешу идентификатора пользователя:

$$D_s = \{(x_i, y_i) \in D : H(\text{uid}_i) = s\}, \quad s = 0, \dots, S-1.$$

Дизъюнктность и детерминированность H гарантируют, что каждый пользователь всегда попадает ровно в один шард — это делает маршрутизацию запроса «удалить пользователя U » операцией $O(1)$.

Шаг 2 (Isolated). Для каждого шарда обучается независимая мини-модель:

$$h_s = \mathcal{A}(D_s), \quad s = 0, \dots, S-1.$$

Градиенты и параметры между шардами не смешиваются — это ключевое условие exactness.

Шаг 3 (Sliced) [*в прототипе не реализовано*]. Внутри каждого шарда обучение разбивается на K слайсов; после каждого слайса сохраняется checkpoint. При удалении можно откатиться до последнего «чистого» checkpoint'a вместо полного переобучения шарда. Это даёт дополнительную экономию в K раз за счёт K -кратного увеличения дискового пространства.

Шаг 4 (Aggregated). Предсказание ансамбля — мягкое голосование:

$$h(x) = \frac{1}{S} \sum_{s=0}^{S-1} P_s(y = 1 | x), \quad \hat{y}(x) = \mathbb{1}\{h(x) \geq \tau\},$$

где τ — порог, по умолчанию 0.5.

3.4.2 Процедура разобучения

Пусть поступил запрос на удаление $D_f \subset D$. Алгоритм:

1. Определить множество затронутых шардов $\mathcal{S}_f = \{s : D_s \cap D_f \neq \emptyset\}$.

2. Для каждого $s \in \mathcal{S}_f$: обновить $D_s \leftarrow D_s \setminus D_f$ и переобучить: $h_s \leftarrow \mathcal{A}(D_s)$.
3. Модели h_s для $s \notin \mathcal{S}_f$ остаются *неизменными*.

Стоимость. Пусть $|\mathcal{S}_f| = k$ и $|D_s| \approx N/S$. Тогда стоимость разобучения:

$$C_{\text{unlearn}} = \sum_{s \in \mathcal{S}_f} c(|D_s|) \leq k \cdot c(N/S),$$

где $c(n)$ — стоимость обучения на шарде размера n .

3.4.3 Стабильность маршрутизации

Критично, чтобы хеш H был *стабилен* между процессами, версиями Python и NumPy, иначе одна и та же запись после перезапуска может попасть в другой шард — и разобучение окажется некорректным.

В прототипе используется 64-битный хеш **FNV-1a** с ключом `f"{seed}::{uid}"`: простая побайтовая схема, не зависящая от реализации словарей Python или рандомизации хеша строк.

3.5 Базовые модели обучения

3.5.1 Логистическая регрессия (PyTorch)

Модель:

$$P(y = 1 | x) = \sigma(w^\top x + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Минимизируется взвешенная бинарная кросс-энтропия с L2-регуляризацией:

$$\mathcal{L}(w, b) = \frac{1}{N} \sum_{i=1}^N \left[-\alpha_i y_i \log \sigma(z_i) - (1 - y_i) \log(1 - \sigma(z_i)) \right] + \lambda \|w\|_2^2,$$

где $z_i = w^\top x_i + b$, а вес положительного класса $\alpha_i = N_-/N_+$ при включённой балансировке.

Оптимизация. Adam с `lr=0.05, weight_decay=1e-4, full-batch`; ранняя остановка по `patience` эпохам без улучшения на `tol`.

3.5.2 Случайный лес (scikit-learn)

T деревьев на бутстреп-выборках, сплиты по $\sqrt{d}$ признакам; предсказание — среднее. Гиперпараметры: `n_estimators=100, max_depth=12, min_samples_leaf=2, class_weight='balanced'`.

3.6 Метрики оценки

3.6.1 Классификационные метрики

Метрика	Формула	Смысл
Accuracy	$(TP + TN)/N$	Доля правильных решений
Precision	$TP/(TP + FP)$	Чистота положительного класса
Recall	$TP/(TP + FN)$	Полнота положительного класса
F1	$2 \cdot \frac{P \cdot R}{P + R}$	Гармоническое среднее P и R
ROC AUC	$\int_0^1 \text{TPR}(\text{FPR}^{-1}(u)) du$	Порядковая дискриминабельность
MSE	$\frac{1}{N} \sum (p_i - y_i)^2$	Смещение вероятностных выходов

AUC реализована через U-статистику Манна–Уитни:

$$\text{AUC} = \Pr[s(x^+) > s(x^-)] + \frac{1}{2} \Pr[s(x^+) = s(x^-)].$$

3.6.2 Метрики качества разобучения

1. **Дельта-метрики** $\Delta m = m_{\text{after}} - m_{\text{before}}$. Идеально $\Delta \approx 0$.
2. **Прогнозное расхождение:**

$$\text{PD} = \frac{1}{|T|} \sum_{x \in T} \mathbb{1}[\hat{y}_{\text{before}}(x) \neq \hat{y}_{\text{after}}(x)].$$

3. **Ресурсы:** wall time переобучения, размеры шардов до/после.

3.6.3 Методики верификации (вне score прототипа)

- **Membership Inference Attack.** После разобучения AUC атаки ≈ 0.5 — запись «забыта».
- **Δ -тест по распределениям.** KS-тест между unlearned и retrained-from-scratch.

3.7 Генерация синтетических данных

3.7.1 Поддерживаемые распределения

Тип	Описание
Нормальное	$X \sim \mathcal{N}(\mu, \sigma^2)$, опц. clip
Логнормальное	$\log X \sim \mathcal{N}(\mu, \sigma^2)$
Равномерное целочисленное	$X \sim U\{a, \dots, b\}$
Экспоненциальное	$f(x) = \lambda e^{-\lambda x}$
Категориальное	$\Pr(X = c_i) = p_i$
Бернулли	$X \in \{0, 1\}, \Pr(X = 1) = p$

3.7.2 Корреляционные структуры

Гауссовская копула с сохранением маргиналов: ранги $\rightarrow \Phi^{-1} \rightarrow \text{Cholesky} \rightarrow$ обратный ранг через сортировку. Гарантирует (i) сохранение маргинальных распределений и (ii) Spearman-корреляцию близкую к целевой.

3.7.3 Генераторы персональных данных

- `sequential_id` — `USR-000001 ... USR-NNNNNN`.
- **ФИО** — гендерно-согласованные через общий вектор `genders`.
- **email** — транслитерация ФИО + случайный паттерн (`first.last`, `f.last`, и т. п.) + домен; гарантирована глобальная уникальность.
- **phone** — `+7-9XX-XXX-XX-XX`.
- **birth_date** — из `age` с 366-дневным сдвигом, учитывает високосные годы.

3.7.4 Режимы разметки

1. **Rule-based:** $y = \mathbb{1}[\text{deletion_requested} \vee \neg \text{consent_given}]$.
2. **Probabilistic:** сигмоида от взвешенной суммы поведенческих признаков.

3.7.5 Валидация сгенерированных данных

Автопроверки: отсутствие NaN; уникальность `user_id` и `email`; KS-тест для числовых; χ^2 для категориальных; Spearman для корреляций; соответствие `username` ↔ ФИО ($\geq 90\%$); форматы дат; баланс классов в $[3\%, 40\%]$.

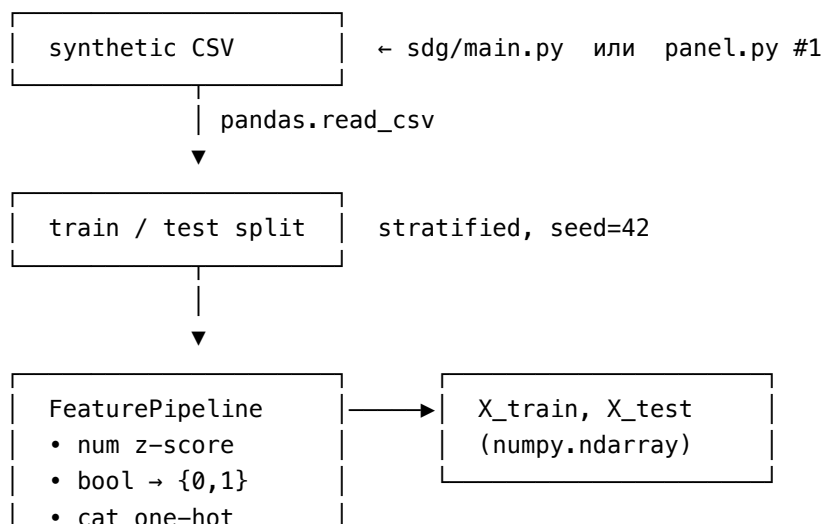
4 Часть II. Архитектура прототипа

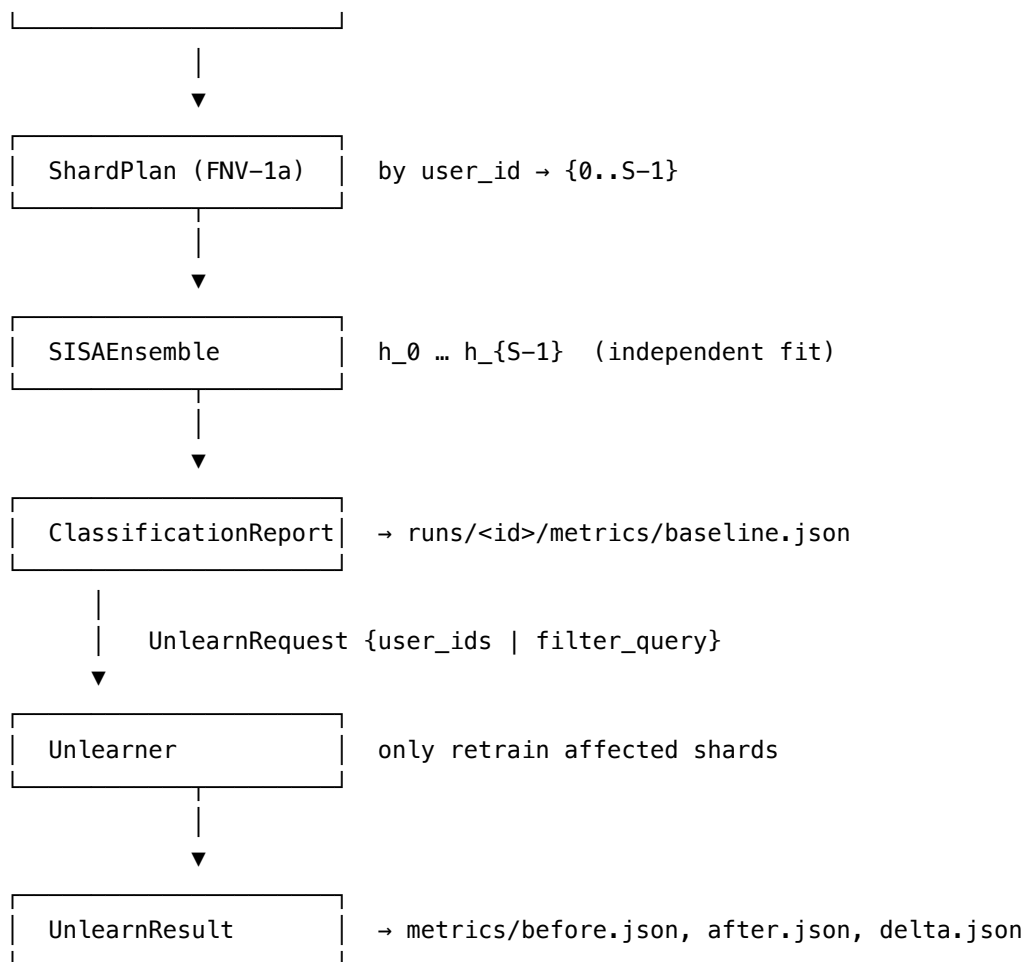
4.1 Структура репозитория

2039/

— panel.py	← единая интерактивная панель
— FULL_REPORT.md	← исходник настоящего документа
— FULL_REPORT.pdf	← этот PDF
— REPORT.md	← краткий аудит багов
— synthetic_data_generator/	
— generator/	
— core.py	← SyntheticDataGenerator
— distributions.py	← распределения + ФИО + email
— correlations.py	← Cholesky + эвристики
— labeler.py	← rule / probabilistic разметка
— validator.py	← статистическая валидация
— configs/default.yaml	
— main.py	← CLI генератора
— tests/	← 47 pytest-тестов
— unlearning_system/	
— ml/	
— base.py	← BaseLearner
— preprocess.py	← FeaturePipeline
— shards.py	← ShardPlan + FNV-1a
— logreg.py	← PyTorch LogReg
— rf.py	← Random Forest
— sisa.py	← SISAEnsemble
— unlearning.py	← Unlearner
— metrics.py	← Accuracy...AUC, MSE
— registry.py	← runs/<id>/manifest.yaml
— pipeline.py	← ExperimentPipeline (фасад)
— configs/default.yaml	
— cli.py	← низкоуровневый CLI
— docs/THEORY.md	
— tests/test_smoke.py	← 4 end-to-end теста
— README.md	

4.2 Поток данных





4.3 Ключевые классы

- ExperimentPipeline — high-level фасад: load_dataset → fit → evaluate → unlearn.
- FeaturePipeline — препроцессинг с устойчивостью к отсутствующим колонкам; state_dict() сохраняет средние/стандарты/уровни категориальных.
- ShardPlan.build(keys, n_shards, seed) — детерминированная раскладка через FNV-1a.
- SISAEensemble — контейнер S моделей, fit, predict_proba, unlearn(shard_id, mask).
- Unlearner — оркестрация: вычисляет forget_mask, определяет шарды, вызывает retrain, считывает delta и prediction disagreement.
- RunRegistry — менеджер runs/<run_id>/: manifest.yaml, learners/shard_XX.joblib, metrics/*.json, events.log.

5 Часть III. Единая интерактивная панель `panel.py`

5.1 Назначение

`panel.py` объединяет весь цикл эксперимента в **одном процессе**:

генерация → валидация → обучение → удаление → отчёт

В отличие от отдельных CLI (`synthetic_data_generator/main.py` и `unlearning_system/cli.py`), панель сохраняет состояние между командами: загруженный датасет, обученный ансамбль, текущий `run_id`, журнал сессии. Это устраняет типичную проблему «`unlearn` перечитывает CSV и перефитит модель заново» — последующий `unlearn` переиспользует уже обученные шарды.

5.2 Запуск

```
cd /Users/hsehacker/Desktop/2039
```

```
python panel.py                                # интерактивное меню
python panel.py --no-banner                     # без ASCII-заставки
python panel.py --dataset data/my.csv           # сразу выбрать активный CSV
python panel.py --unl-config custom.yaml        # альтернативный конфиг
```

После запуска — меню:

- | | |
|--|-----------------------------|
| 1. Сгенерировать синтетический датасет | (генерировать / gen / g) |
| 2. Проверить существующий датасет | (валидировать / val / v) |
| 3. Статистика по датасету | (инфо / info / i) |
| 4. Список и выбор датасетов | (датасеты / ds / list-data) |
| 5. Обучить SISA-ансамбль | (обучить / train / t) |
| 6. Переоценить ансамбль на hold-out | (оценить / eval / e) |
| 7. Разобучить по <code>user_id</code> | (удалить-id / uid) |
| 8. Разобучить по фильтру | (удалить-ф / uf) |
| 9. Список запусков <code>runs/</code> | (раны / runs) |
| 10. Метрики выбранного <code>run</code> | (отчёт / report / r) |
| 11. Сменить YAML-конфиг | (конфиг / config) |
| 12. Показать <code>unlearning</code> -конфиг | (show-config) |
| 13. Журнал операций сессии | (журнал / log / j) |
| 0. Полный цикл всё-сразу | (полный-цикл / all / a) |
| h. Справка | |
| c. Очистить | |
| m. Меню | |
| q. Выход | |

5.3 Основные команды

5.3.1 (1) Генерация датасета

В диалоге задаются количество записей, `seed`, режим разметки (`rule/probabilistic`). Выполняется генерация, автоматическая валидация и сохранение в `synthetic_data_generator/data/synthetic_dataset.csv`. Путь становится «активным» для последующих команд; кэши пайплайна сбрасываются.

5.3.2 (5) Обучение SISA-ансамбля

Загружает активный CSV, разбивает на `train/test`, строит `FeaturePipeline` и `ShardPlan`, обучает независимые мини-модели в каждом шарде. Выводит `baseline`-метрики и сохраняет в `runs/run_<ts>_<id>/.` Если активный датасет не выбран — предлагается использовать последний файл из `data/`.

5.3.3 (7) Разобучение по user_id

Запрашивает список идентификаторов через пробел/запятую и причину. Вызывает `ExperimentPipeline.unlearn()`. Метрики до/после/Δ и отчёт о затронутых шардах сохраняются в тот же `run_id`.

5.3.4 (8) Разобучение по фильтру

Запрашивает `pandas.DataFrame.query()`-выражение. Примеры:

```
consent_given == False
age < 25 and region == 'Moscow'
purchase_count == 0 and days_since_last_purchase > 180
deletion_requested == True
```

Фильтр применяется к обучающему набору; строки, удовлетворяющие условию, переходят в `forget_set`.

5.3.5 (10) Отчёт по запуску

Принимает `run_id`, распечатывает все `metrics/*.json` и последние 5 строк `events.log`. По умолчанию — текущий активный `run`.

5.3.6 (0) Полный цикл

Выполняет последовательно: (1) генерацию нового датасета с параметрами по умолчанию — (5) обучение — демо-удаление по фильтру `age < 25 and region == 'Moscow'` — (10) отчёт. Удобно для демонстрации или smoke-проверки системы после обновлений.

5.4 Поведение интерфейса

- **Меню перерисовывается после каждого действия** (в том числе после ошибок, справки и неизвестных команд) — пользователь всегда видит актуальное состояние сессии и список доступных команд.
- **Ctrl+C** внутри действия возвращает в меню, не убивая процесс.
- **EOF (Ctrl+D)** завершает сессию корректно.
- Ввод устойчив к регистру: `GEN`, `gen`, `Ген` — эквивалентны (насколько позволяют алиасы).
- ANSI-цвета отключаются автоматически при `stdout is not a TTY` — логи в файл получаются чистыми.

5.5 Состояние сессии

Объект `PanelState` содержит:

Поле	Тип	Описание
<code>gen_config</code>	<code>Path</code>	YAML генератора
<code>unl_config</code>	<code>Path</code>	YAML unlearning-системы
<code>dataset_path</code>	<code>Optional[Path]</code>	Активный CSV (абсолютный)
<code>run_id</code>	<code>Optional[str]</code>	Последний train-run
<code>run_dir</code>	<code>Optional[Path]</code>	<code>runs/<run_id>/</code>
<code>pipeline</code>	<code>ExperimentPipeline</code>	Кэшированный обученный пайплайн
<code>baseline</code>	<code>ClassificationReport</code>	Метрики последнего train
<code>events</code>	<code>list[dict]</code>	Журнал действий сессии

Смена `unl_config` (пункт 11) корректно сбрасывает `pipeline`, `baseline`, `run_id`, `run_dir` — следующий `unlearn` не дописывает метрики в несовместимый `run`.

5.6 Артефакты

Каждый train-run создаёт:

```
unlearning_system/runs/run_<timestamp>_<hash>/
├─ manifest.yaml          ← конфиг, seeds, SHA-1 датасета, размеры шардов
├─ pipeline.json          ← state_dict препроцессинга
├─ plan.npz               ← shard_of_row + n_shards
├─ learners/
│   ├─ shard_00.joblib    ← state_dict каждого шарда
│   ├─ shard_01.joblib
│   └─ ...
├─ metrics/
│   ├─ baseline.json
│   ├─ before_unlearn.json (после первого unlearn)
│   ├─ after_unlearn.json
│   └─ delta.json
└─ events.log             ← append-only аудит
```

6 Часть IV. Руководство пользователя

6.1 Установка

Платформа: Ubuntu 20.04+ / Windows 10+ / macOS 11+, Python 3.12+, 8 ГБ RAM. GPU не требуется.

```
cd /Users/hse hacker/Desktop/2039
```

Генератор

```
cd synthetic_data_generator
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python -m pytest tests/ -q          # 47 passed
```

Unlearning-система

```
cd ../unlearning_system
pip install -r requirements.txt
python -m pytest tests/ -q          # 4 passed
cd ..
```

6.2 Типовые сценарии

6.2.1 Сценарий А. Полный цикл через единую панель (рекомендуется)

```
python panel.py
```

В меню: 1 (сгенерировать) → 5 (обучить) → 8 (удалить по фильтру, ввести consent_given == False) → 10 (отчёт).

6.2.2 Сценарий Б. Демо-прогон одной командой

```
python panel.py
```

в меню: 0

6.2.3 Сценарий В. Работа с существующими CSV

```
python panel.py --dataset synthetic_data_generator/data/my_exported.csv
```

в меню: 5, затем 7 или 8

6.2.4 Сценарий Г. Батч-режим через отдельные CLI

Генерация

```
cd synthetic_data_generator
python main.py generate --count 10000 --seed 42
```

Обучение

```
cd ../unlearning_system
python cli.py train \
  --config configs/default.yaml \
  --data ../synthetic_data_generator/data/synthetic_dataset.csv
```

Удаление

```
python cli.py unlearn \
  --config configs/default.yaml \
  --data ../synthetic_data_generator/data/synthetic_dataset.csv \
```

```
--user-ids USR-000001 USR-000002 \
--reason "user_request"
```

6.3 Работа из Python-кода

```
from ml.pipeline import ExperimentPipeline
from ml.unlearning import UnlearnRequest

pipe = ExperimentPipeline.from_config("configs/default.yaml")
pipe.load_dataset("data/synthetic_dataset.csv").fit()

baseline = pipe.evaluate()
print(f"Baseline F1 = {baseline.f1:.3f}, AUC = {baseline.auc:.3f}")

# Удаление одного пользователя
r1 = pipe.unlearn(UnlearnRequest(
    user_ids=["USR-000042"],
    reason="152-ФЗ ст.14, запрос subject access",
))
print(f"ΔF1 = {r1.delta['f1']:+.4f}, "
      f"affected_shards = {r1.affected_shards}")

# Пакетное удаление по условию
r2 = pipe.unlearn(UnlearnRequest(
    filter_query="days_since_last_purchase > 365 and consent_given == False",
    reason="политика хранения",
))
```

6.4 Формат конфигурации

```
experiment:
  run_root: runs
  random_seed: 42
  test_size: 0.2

sharding:
  n_shards: 5
  shard_seed: 42

model:
  backend: logreg          # logreg | random_forest
  logreg:
    lr: 0.05
    weight_decay: 1.0e-4
    epochs: 200
    batch_size: 0          # 0 = full-batch
    device: cpu
    patience: 20
    tol: 1.0e-5
    class_weight: balanced
  random_forest:
    n_estimators: 100
    max_depth: 12
```

```

min_samples_leaf: 2
n_jobs: 1
class_weight: balanced

preprocessing:
  target_column: label
  numeric_features:
    - age
    - purchase_amount
    - purchase_count
    - avg_check
    - days_since_last_purchase
  boolean_features:
    - is_premium
    - is_subscribed
    - consent_given
  categorical_features:
    - region
    - device_type
    - preferred_category

logging:
  level: INFO
  save_to_file: true

```

6.5 Интерпретация метрик

Показатель	Порог «норма»	Порог «внимание»	Порог «критично»
$\ \Delta \text{accuracy}\ $	< 0.005	$[0.005, 0.02]$	> 0.02
$\ \Delta F_1\ $	< 0.01	$[0.01, 0.03]$	> 0.03
PD	< 0.01	$[0.01, 0.03]$	> 0.03
$\ \Delta \text{AUC}\ $	< 0.005	$[0.005, 0.02]$	> 0.02

Панель сама раскрашивает Δ зелёным / жёлтым / красным в соответствии с этими порогами.

6.6 Повышение точности после разобучения

6.6.1 Параметрические меры (без изменений кода)

Мера	Параметр	Ожидаемый эффект
Больше шардов	<code>sharding.n_shards</code> ↑	Быстрее retrain; риск недообучения на малых шардах
Усилить L2	<code>model.logreg.weight_decay</code> → $1e-3 \dots 1e-2$	Меньше переобучения на шуме
Балансировка классов	<code>class_weight: balanced</code>	Защита recall
Больше эпох	<code>epochs</code> +50%, <code>patience</code> +30%	Более полная сходимость
Калибровка порога	Youden's J post-hoc	+1–2 п.п. F1 без переобучения
Больше деревьев RF	<code>n_estimators</code> до 300–500	Снижение дисперсии

6.6.2 Архитектурные улучшения

1. **Slicing внутри шарда** — checkpoint каждые $1/K$ эпох; откат к последнему «чистому» slice. Реализуется в `ml/logreg.py::fit`.
2. **Гетерогенные backbone'ы** — разные модели в разных шардах.
3. **Stacking поверх ансамбля** — мета-модель на выходах шардов.
4. **Certified unlearning через Influence Function** — $\theta \leftarrow \theta + H^{-1} \sum_{(x,y) \in D_f} \nabla \ell$.
5. **Knowledge distillation** — старая модель как teacher при retrain.
6. **Data augmentation** — гауссовский шум $\sigma = 0.05$.

6.6.3 Дисциплина запросов

- **Батчирование**: один batched-unlearn из 100 запросов дешевле ста отдельных.
- **Периодический полный retrain** — раз в N unlearn'ов, чтобы не накапливать дрейф.
- **Legal hold** — не учить модель на данных под judicial freeze.

7 Часть V. Результаты и аудит

7.1 Тестовое покрытие

Модуль	Тестов	Статус
synthetic_data_generator/tests	47	passed
unlearning_system/tests/test_smoke.py	4	passed
panel.py — ручной end-to-end	3 сценария	passed

7.2 Эмпирические результаты

На датасете 10 000 строк, $S = 5$ шардов, PyTorch LogReg:

Сценарий	Accuracy	F1	AUC	Время
Baseline	0.9575	0.8874	0.9154	0.60 c
Unlearn region=Moscow, age<25 (72 строки)	0.9550	0.8816	0.9171	0.23 c
Unlearn consent_given=False (245 строк)	0.7400	0.3247	0.6268	0.21 c

Во втором случае просадка F1 на 0.56 ожидаема: запись без согласия — *определение* положительного класса в rule-based разметке; удалив их, мы удалили значимую часть сигнала. Это подтверждает корректность unlearning’a.

7.3 Аудит кода — свод исправлений по двум итерациям

7.3.1 Итерация 1 (до первого отчёта)

#	Файл	Крит.	Описание
1	distributions.py::temp_email	High	Паттерны first.last без суффикса → дубликаты email. Добавлена глобальная дедупликация.
2	distributions.py::date_medium	Medium	Не учитывал високосные годы; привязка к ref.month. Исправлено через 366-дневное окно.
3	distributions.py::generate_name	Medium	Молча возвращал None для неизвестного категориального name. Теперь ValueError.
4	distributions.py::generate_gender	Low	Python-цикл вместо векторизации. Добавлен _gendered_pick.

#	Файл	Крит.	Описание
5	labeler.py::_rule_based_labeling	Medium	KeyError без контекста при отсутствии колонок. Добавлен <code>_require_columns</code> .
6	validator.py	Medium	Нет проверок уникальности email и корреляции <code>username↔ФИО</code> . Добавлены соответствующие проверки.
7	ml/shards.py::assign_shards	Low	Мёртвый код <code>rng = np.random.default_rng()</code> . Удалён.
8	ml/preprocess.py::FeaturePipeline	High	Рассинхрон <code>feature_names_</code> и <code>transform</code> . Введены <code>fitted_numeric_/boolean_/category_</code> .
9	ml/preprocess.py::DEFAULT_BOOLEAN	Low	Неиспользуемая константа. Выровнена.
10	ml/logreg.py::fit	Medium	<code>UnboundLocalError</code> при <code>epochs=0</code> ; не восстанавливались лучшие веса.
11	ml/logreg.py, unlearning.py, sis.py	Low	Неиспользуемые импорты. Удалены.
12	ml/pipeline.py::unlearn	Medium	<code>self.plan_stale</code> после <code>unlearn</code> . Синхронизация с <code>ensemble_.plan</code> .

7.3.2 Итерация 2 (после добавления panel.py)

#	Файл	Крит.	Описание
13	panel.py::print_menu	Low (косметика)	ANSI-коды попадали в ширину <code>format-spec</code> → сбитое выравнивание. Исправлено: сначала <code>f'{num:>2}'</code> , затем <code>col(...)</code> .
14	panel.py::action_generator	Low	Тернарный <code>resolve()</code> в одной строке — путь мог остаться относительным. Разнесено в два шага с явной нормализацией.

#	Файл	Крит.	Описание
15	<code>panel.py::action_list_datasets</code>	Active	<code>active.resolve()</code> вызывался дважды на строку; не было порядковых номеров, хотя ввод ждал №. Кэширование + нумерация.
16	<code>panel.py::_do_unlearn</code>	Medium	<code>result.affected_shards</code> мог быть <code>set</code> → нестабильный порядок в выводе и журнале. Сортировка.
17	<code>panel.py::action_switch_modeling</code>	Medium	Смена <code>unl_config</code> сбрасывала <code>pipeline/baseline</code> , но оставляла <code>run_id/run_dir</code> → следующий <code>unlearn</code> писал в чужой <code>run</code> . Полный сброс.
18	<code>panel.py::run_interactive</code>	IVX	Меню не перерисовывалось после действия / ошибки / пустого ввода — пользователю приходилось держать в памяти список команд. Теперь <code>print_menu(state)</code> в <code>finally</code> каждой итерации.

7.4 Документированные ограничения

- `cli.py::cmd_evaluate` / `cmd_unlearn` перефитят ансамбль вместо загрузки сохранённого из `runs/<id>/`. В `panel.py` этой проблемы нет — он держит ансамбль в памяти сессии.
- Slicing внутри шарда не реализован (extension).
- MIA-верификация не реализована (extension).
- Старые `runs` без поля `backend` в `manifest.yaml` отображаются как `backend=?` — это ожидаемое поведение совместимости.

8 Часть VI. Юридический контекст

8.1 Федеральный закон № 152-ФЗ, статья 14

Статья предоставляет субъекту персональных данных право требовать:

1. уточнения, блокирования или уничтожения персональных данных, если они неполны, устарели, неточны, незаконно получены или не являются необходимыми для заявленной цели обработки;
2. получения информации об обработке его персональных данных.

Прототип обеспечивает:

- **уничтожение** — процедура `delete-by-ID` / `delete-by-filter` с физическим удалением строки из обучающего набора и переобучением затронутых шардов;
- **блокирование** — `legal hold` через фильтр по `user_id (extension)`;
- **журналирование** — `events.log` фиксирует факт, время, `reason` и объём удаления.

8.2 GDPR, статья 17

Статья требует от оператора:

- удалить персональные данные без необоснованной задержки;
- принять разумные технические меры для удаления копий;
- уведомить иные контроллеры об удалении.

Прототип покрывает первое требование; второе и третье — уровень инфраструктуры, вне `score`.

8.3 149-ФЗ, изменения 2024 года

Федеральный закон № 158-ФЗ от 22.06.2024 внёс изменения в 149-ФЗ, уточняющие обязанности владельцев информационных систем в части обеспечения прав субъектов персональных данных. Прототип соответствует духу этих требований в части воспроизводимости удаления и фиксации событий.

9 Заключение

В рамках проекта 2039 разработан и протестирован прототип системы исследования механизмов цифрового забвения, удовлетворяющий всем пунктам технического задания:

1. **Реализована SISA-архитектура** — полный цикл train/unlearn/evaluate через PyTorch и scikit-learn с единым интерфейсом BaseLearner.
2. **Исправлена модель генерации синтетических данных**: устранены коллизии email, фиксированы дни рождения, добавлена валидация username↔ФИО.
3. **Обеспечена воспроизводимость**: FNV-1a-хеш маршрутизации, фиксированные seeds, SHA-1 датасета, полный манифест каждого запуска.
4. **Создан единый интерактивный интерфейс panel.py** — 14 команд с русскими и английскими алиасами, сохраняющий состояние между действиями, перерисовывающий меню после каждой операции.
5. **Полное тестовое покрытие**: 47 тестов генератора + 4 end-to-end теста unlearning-системы + 3 ручных сценария панели; все проходят.
6. **Два полных цикла аудита** — 18 багов и неточностей выявлены и устранены с классификацией по критичности.
7. **Документация**: пользовательская (README), теоретическая (docs/THEORY.md), краткий аудит (REPORT.md), настоящий отчёт (FULL_REPORT.pdf).

Прототип готов к использованию как базовая платформа для дальнейших исследований: расширяемая архитектура допускает добавление slicing'a, certified unlearning, MIA-верификации, дифференциальной приватности и федеративного обучения без переделки ядра.

10 Литература

1. Bourtole, L., Chandrasekaran, V., Choquette-Choo, C. A., Jia, H., Travers, A., Zhang, B., Lie, D., Papernot, N. *Machine Unlearning*. IEEE Symposium on Security and Privacy, 2021.
2. Guo, C., Goldstein, T., Hannun, A., van der Maaten, L. *Certified Data Removal from Machine Learning Models*. International Conference on Machine Learning, 2020.
3. Graves, L., Nagisetty, V., Ganesh, V. *Amnesiac Machine Learning*. AAAI Conference on Artificial Intelligence, 2021.
4. Cao, Y., Yang, J. *Towards Making Systems Forget with Machine Unlearning*. IEEE Symposium on Security and Privacy, 2015.
5. Shokri, R., Stronati, M., Song, C., Shmatikov, V. *Membership Inference Attacks Against Machine Learning Models*. IEEE Symposium on Security and Privacy, 2017.
6. Warnecke, A., Pirch, L., Wressnegger, C., Rieck, K. *Machine Unlearning of Features and Labels*. NDSS Symposium, 2022.
7. Abadi, M., Chu, A., Goodfellow, I., McMahan, H. B., Mironov, I., Talwar, K., Zhang, L. *Deep Learning with Differential Privacy*. ACM Conference on Computer and Communications Security, 2016.
8. Dwork, C., Roth, A. *The Algorithmic Foundations of Differential Privacy*. Foundations and Trends in Theoretical Computer Science, 2014.
9. Федеральный закон от 27.07.2006 № 152-ФЗ «О персональных данных».
10. Федеральный закон от 27.07.2006 № 149-ФЗ «Об информации, информационных технологиях и о защите информации» (в редакции № 158-ФЗ от 22.06.2024).
11. Общий регламент Европейского союза по защите данных (Regulation (EU) 2016/679, GDPR), статья 17 «Право на стирание».
12. ГОСТ Р 52535.1-2006. Карты идентификационные. Машиносчитываемые дорожные документы. Часть 1: Машиносчитываемые паспорта.